

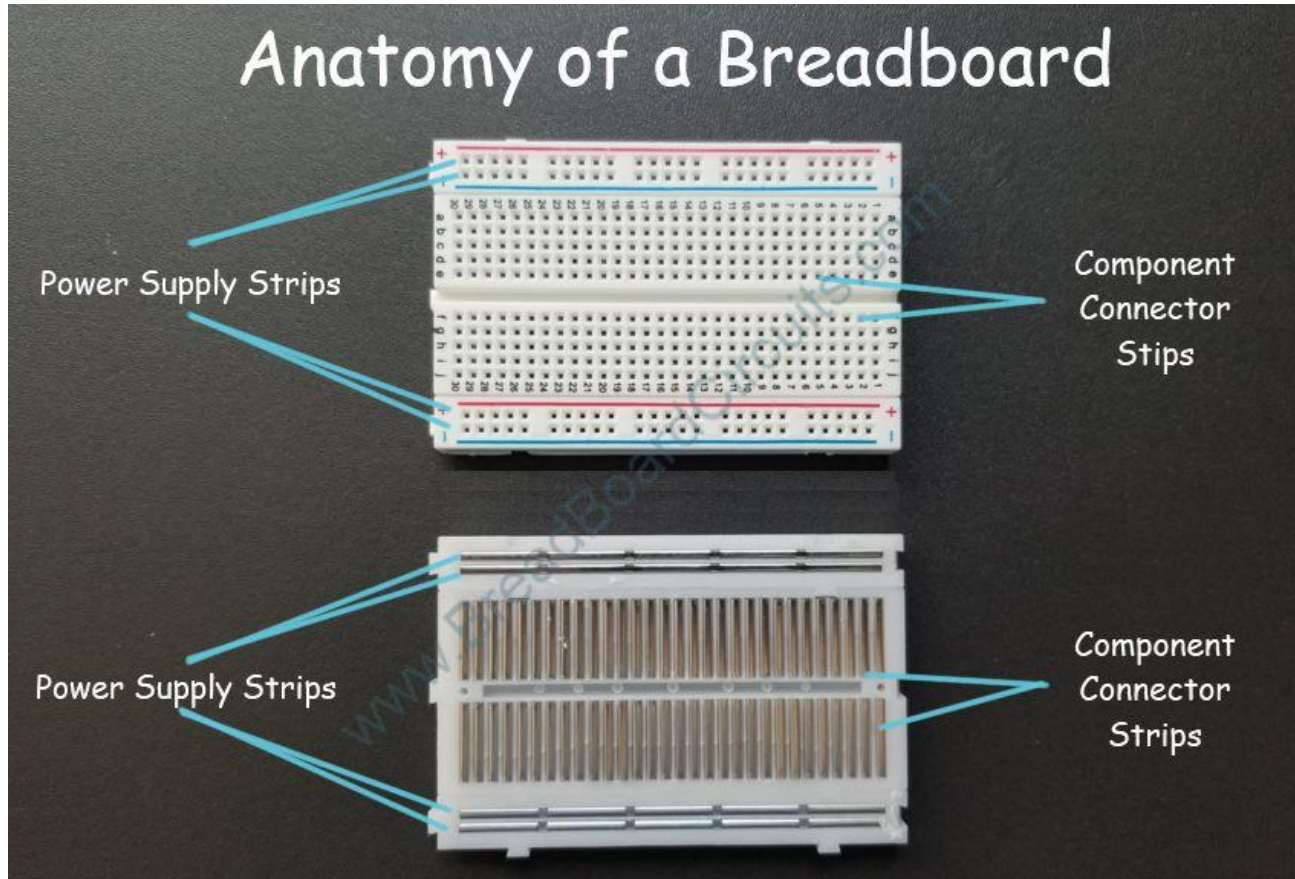
Session 1 Topics

Session 1

Working with ESP32

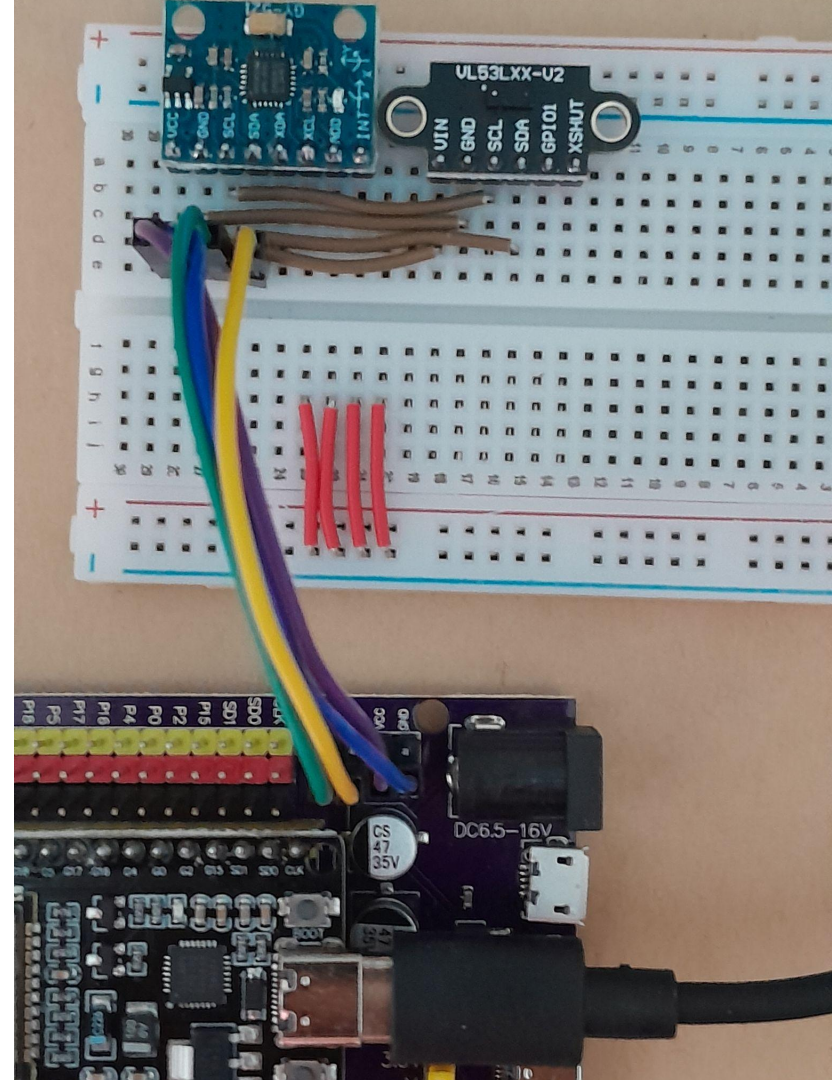
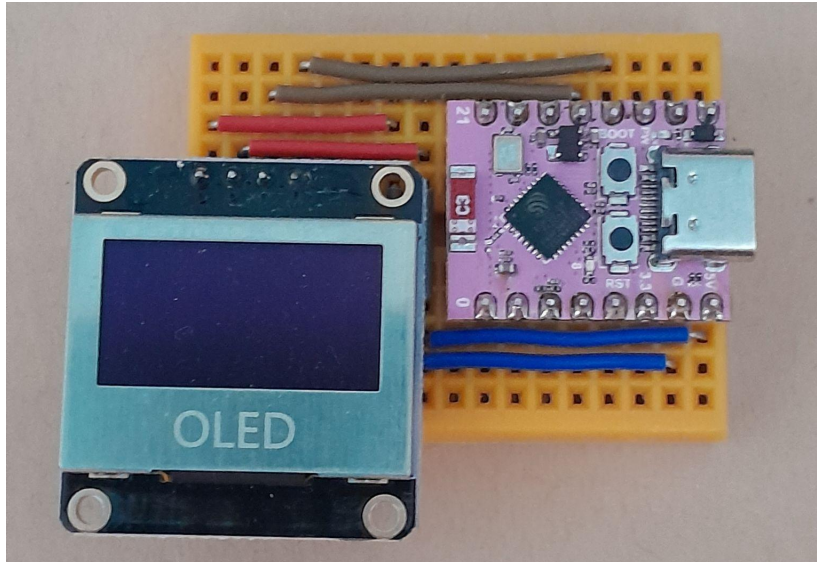
- Quick recap + Intro to electronics
- Working with ESP32, coding + upload
- Input components
 - Dip switch & micro switch
 - Push Button
 - Rotary encoder
 - Joystick
- Making simple sounds with the buzzer

Intro to electronics



Intro to electronics

Jumper wires are used on the breadboard to connect different components to each other.

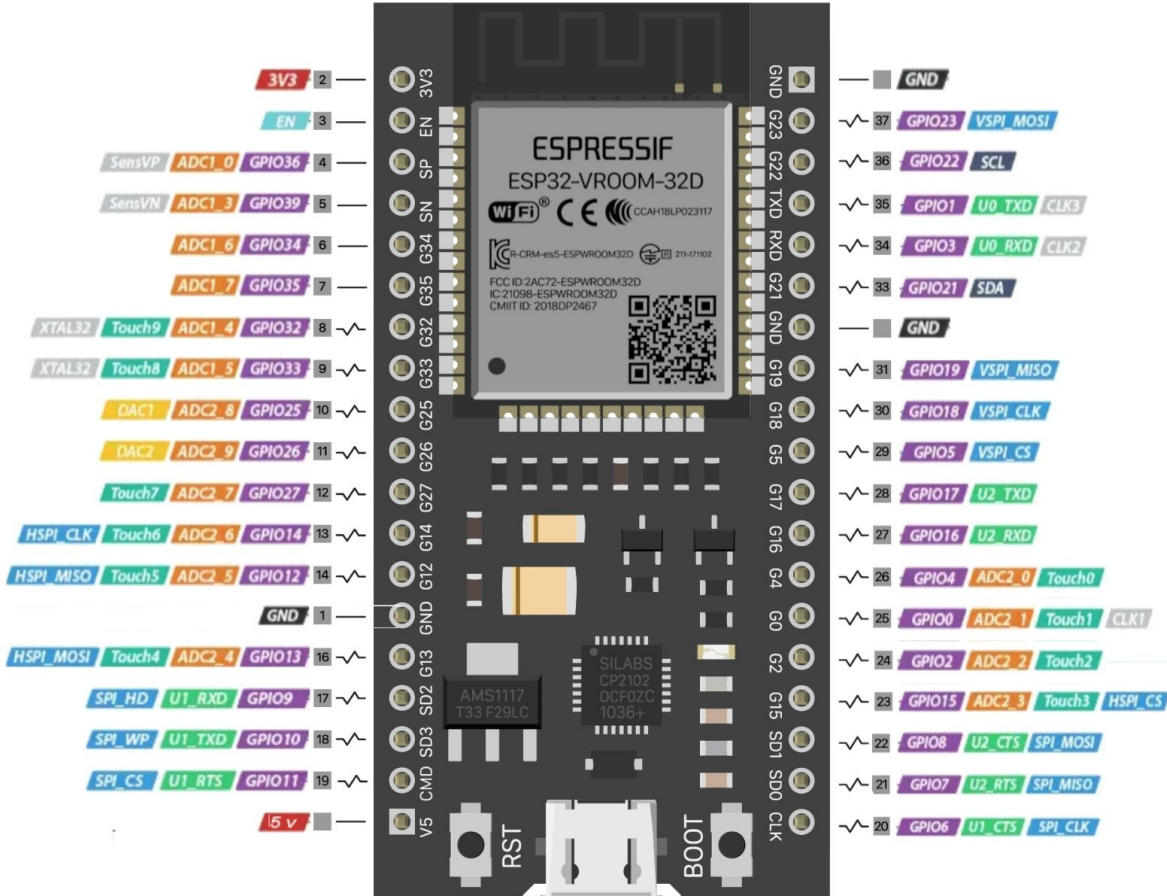


Working with ESP32

Each pin has multiple functions

Match the sensor or device you want to connect with the corresponding pin

E.g, a servo (type of motor) requires a PWM-enabled pin.



Working with ESP32

Important terms:

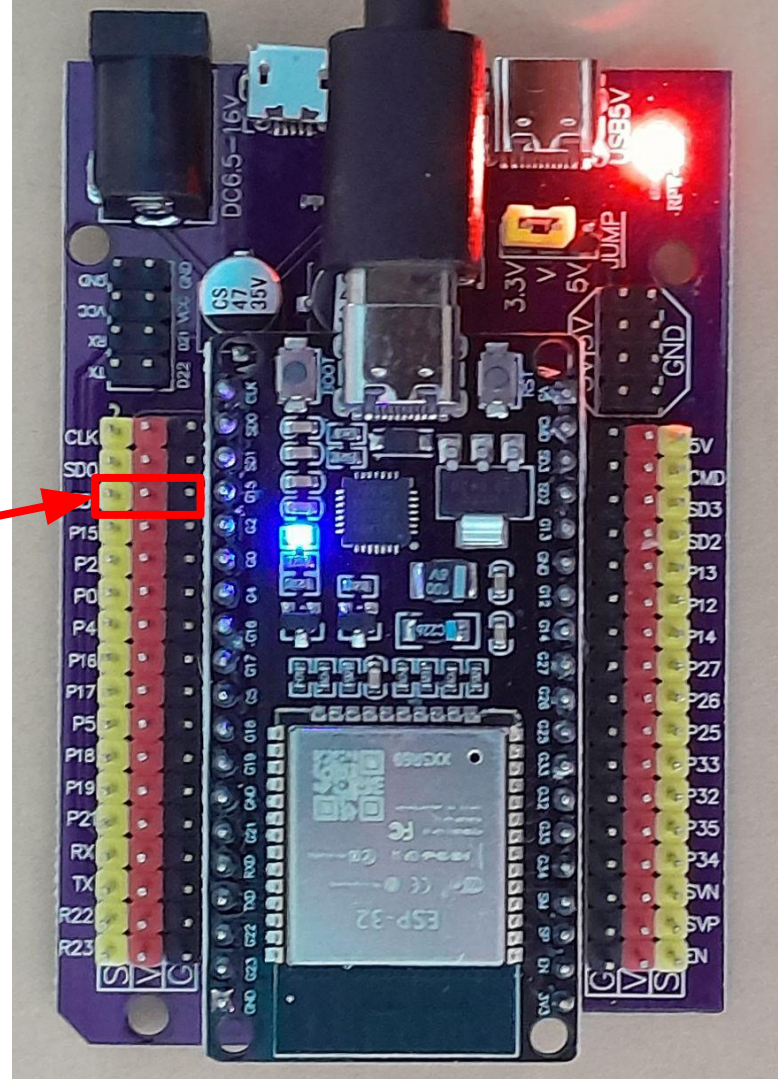
VCC = Power = either 5V or 3.3V = Positive(+) end of power source

GND = Ground = 0V = Negative(-) end of power source

Working with ESP32

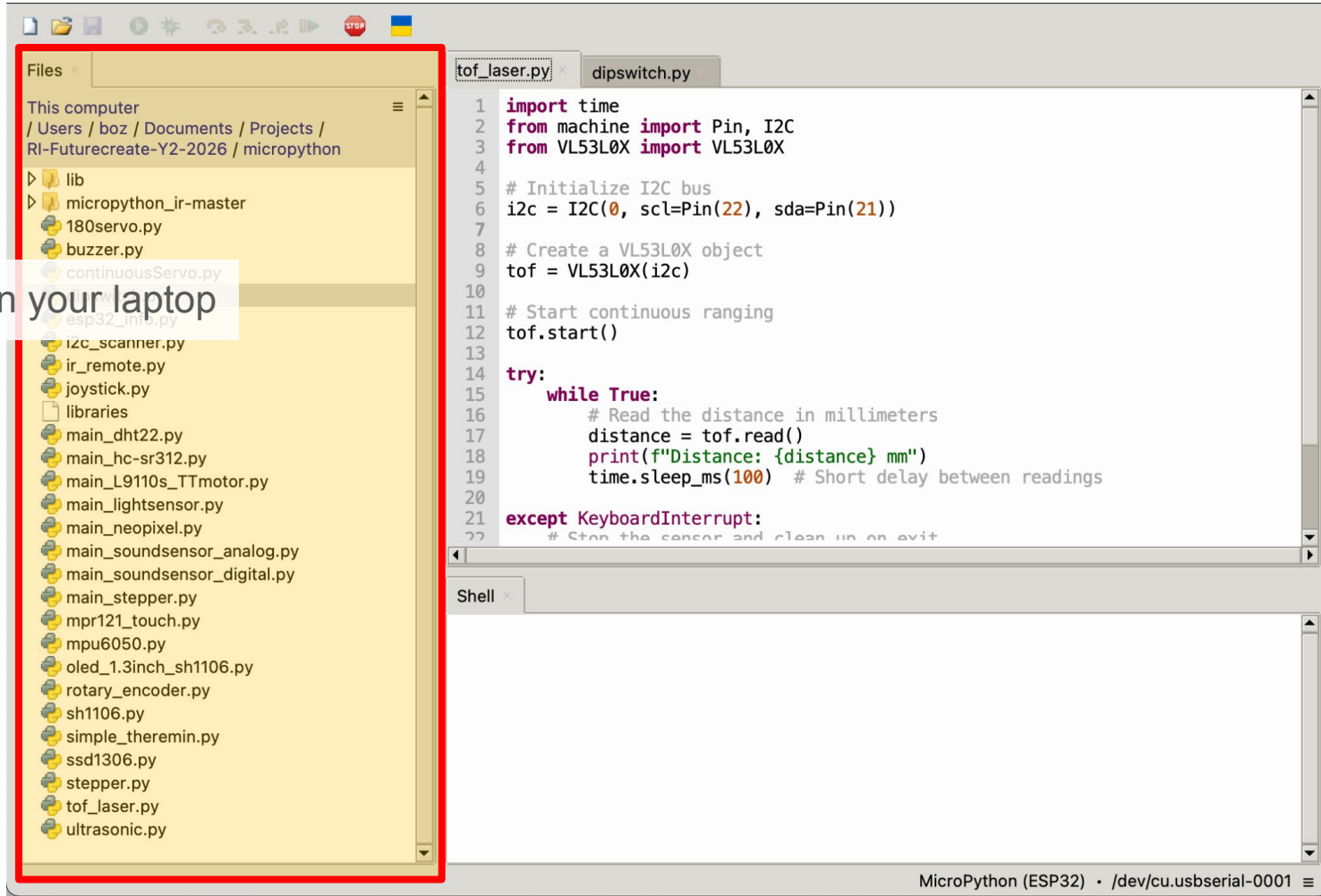
Esp32 breakout board

- Presents GPIO pins as Signal(yellow), Power(red) and Ground(Black)
- Most sensors and devices can be connected in this 3-pin configuration

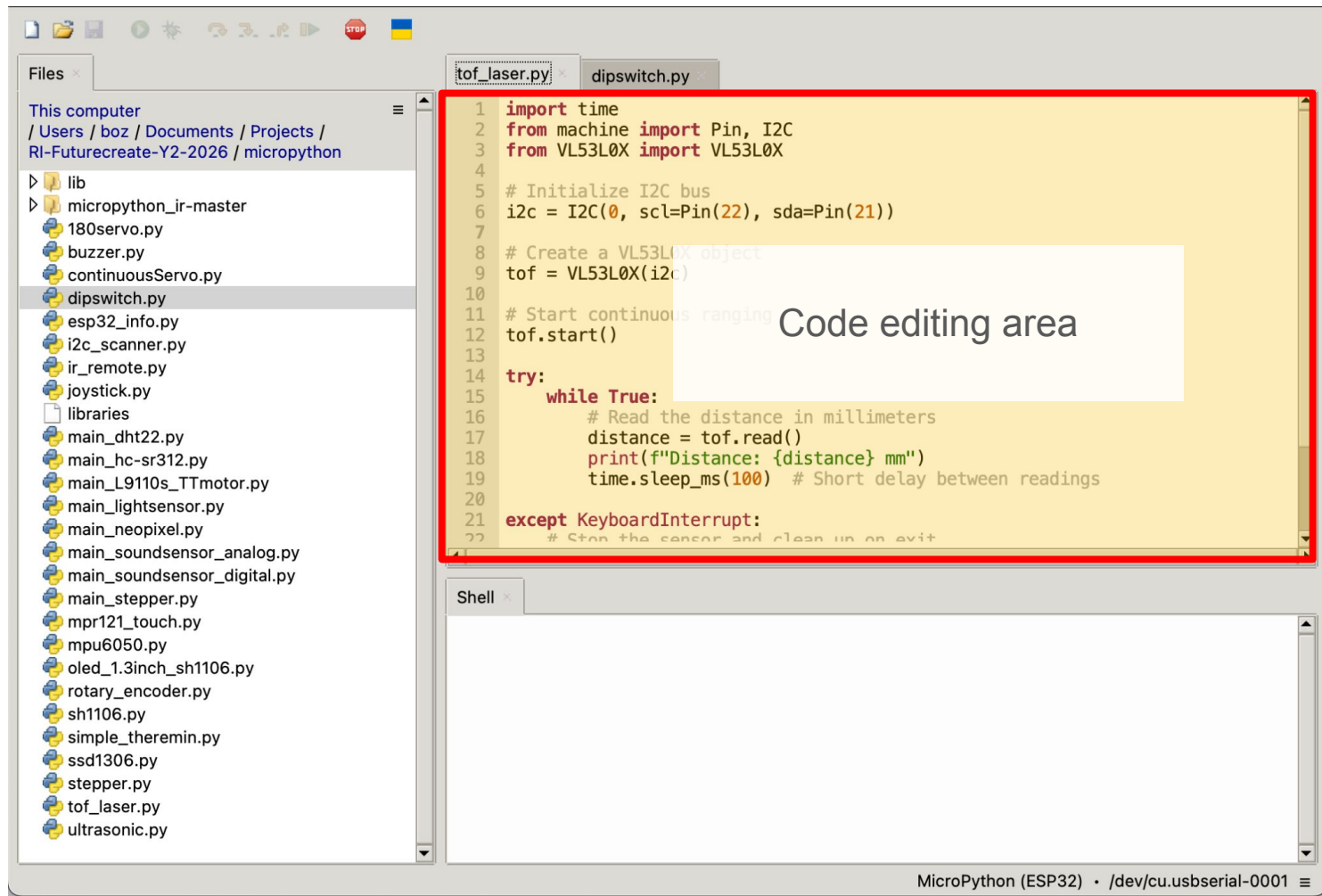


Intro to Thonny IDE

Files on your laptop

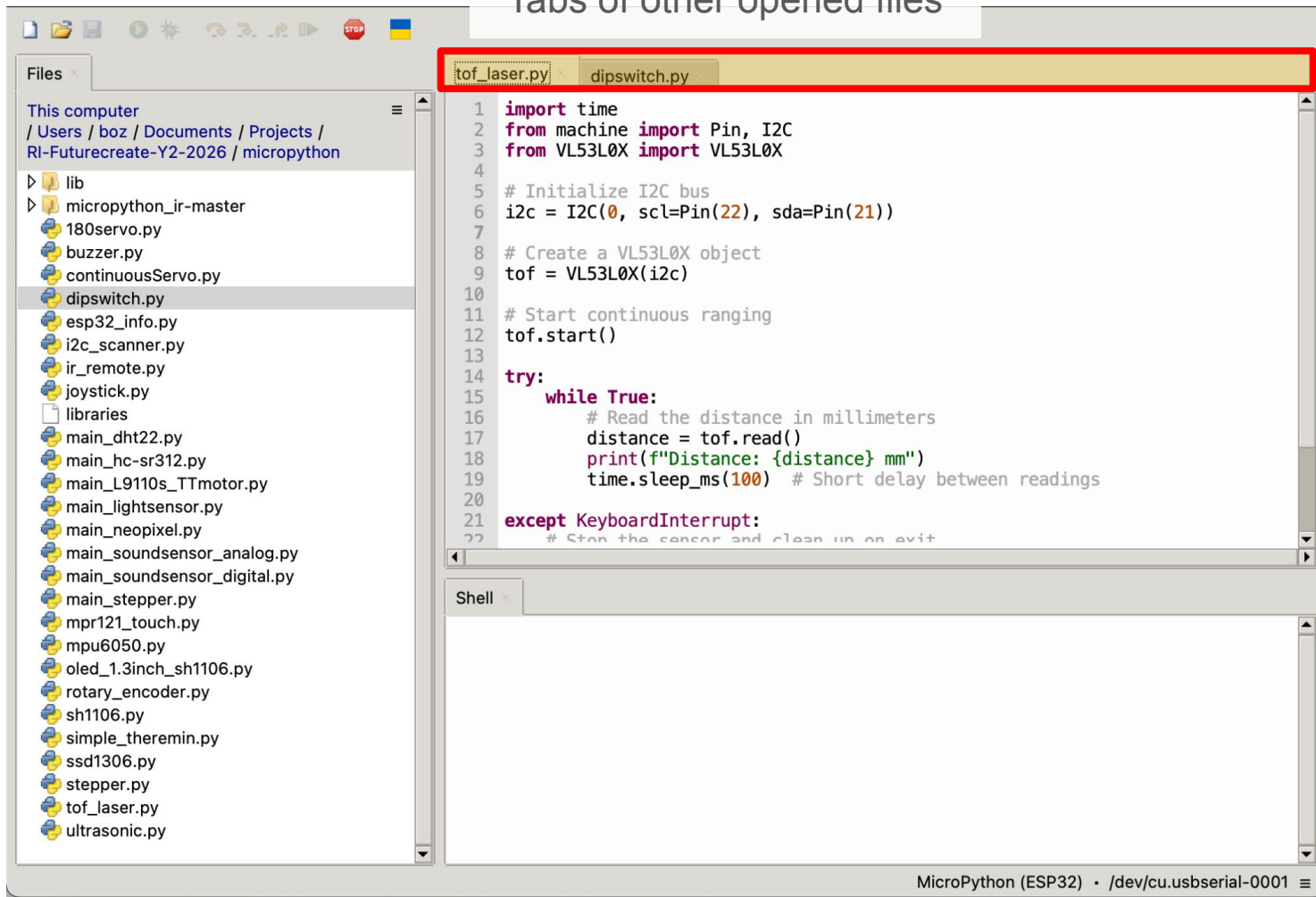


Intro to Thonny IDE



Intro to Thonny IDE

Tabs of other opened files



Intro to Thonny IDE

The screenshot displays the Thonny IDE interface. The top toolbar contains icons for file operations and execution. A red box highlights the 'Run' button (a green play icon). A red arrow points from this button to a larger, detailed view of the toolbar on the left, which also shows the 'Run' button. Below this view, the text 'Run script in active window' is displayed with an arrow pointing to the 'Run' button in the main toolbar. The main editor window shows a Python script named 'tof_laser.py' with the following code:

```
1 import time
2 from machine import Pin, I2C
3 from VL53L0X import VL53L0X
4
5 # Initialize I2C bus
6 i2c = I2C(0, scl=Pin(22), sda=Pin(21))
7
8 # Create a VL53L0X object
9 tof = VL53L0X(i2c)
10
11 # Start continuous ranging
12 tof.start()
13
14 try:
15     while True:
16         # Read the distance in millimeters
17         distance = tof.read()
18         print(f"Distance: {distance} mm")
19         time.sleep_ms(100) # Short delay between readings
20
21 except KeyboardInterrupt:
22     # Stop the sensor and clean up on exit
```

The bottom panel shows the 'MicroPython device' section with a file tree containing 'lib' and 'boot.py'. The 'Shell' window at the bottom right shows the output of the script execution:

```
MPY: soft reboot
MicroPython v1.26.1 on 2025-09-11; Generic ESP32 module with ESP32
Type "help()" for more information.
>>>
```

A red box highlights the Shell window, and a white text box within it contains the text 'When ESP32 is connected...'. The status bar at the bottom indicates 'MicroPython (ESP32) • CP2102 USB to UART Bridge Controller @ /dev/cu.usbserial-0001'.

Intro to Thonny IDE

The screenshot displays the Thonny IDE interface. The top toolbar contains icons for file operations and execution, with a red box highlighting the 'Run' (green play button) and 'Stop' (red stop button) icons. A red arrow points from the 'Stop' icon to a callout box. The callout box shows a zoomed-in view of the toolbar with the text 'Stop and reset ESP32' and an arrow pointing to the 'Stop' icon. The main editor window shows a Python script named `tof_laser.py` with the following code:

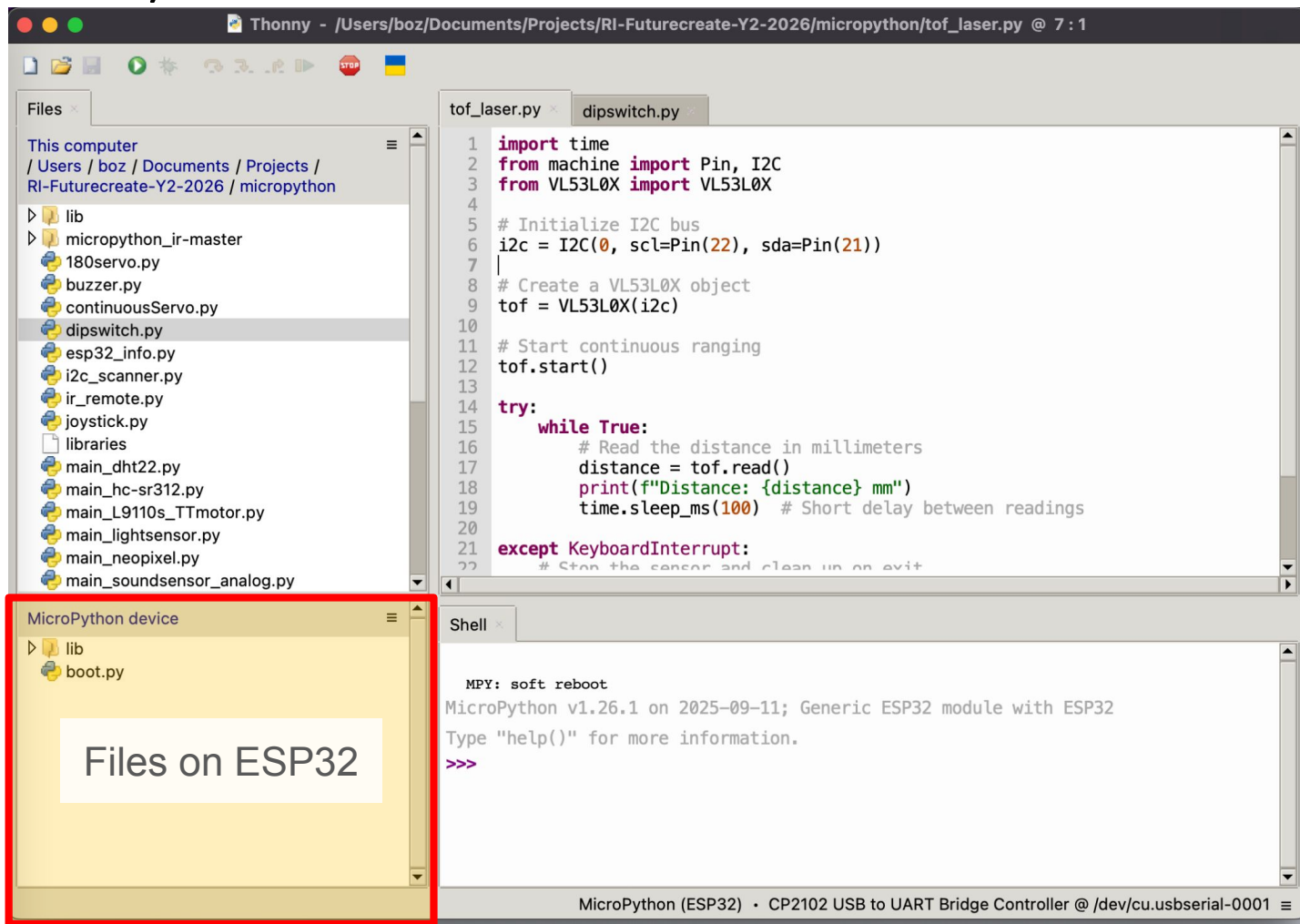
```
1 import time
2 from machine import Pin, I2C
3 from VL53L0X import VL53L0X
4
5 # Initialize I2C bus
6 i2c = I2C(0, scl=Pin(22), sda=Pin(21))
7
8 # Create a VL53L0X object
9 tof = VL53L0X(i2c)
10
11 # Start continuous ranging
12 tof.start()
13
14 try:
15     while True:
16         # Read the distance in millimeters
17         distance = tof.read()
18         print(f"Distance: {distance} mm")
19         time.sleep_ms(100) # Short delay between readings
20
21 except KeyboardInterrupt:
22     # Stop the sensor and clean up on exit
```

The bottom panel shows the 'MicroPython device' section with a file tree containing `lib` and `boot.py`. The 'Shell' window at the bottom right shows the output of the script execution:

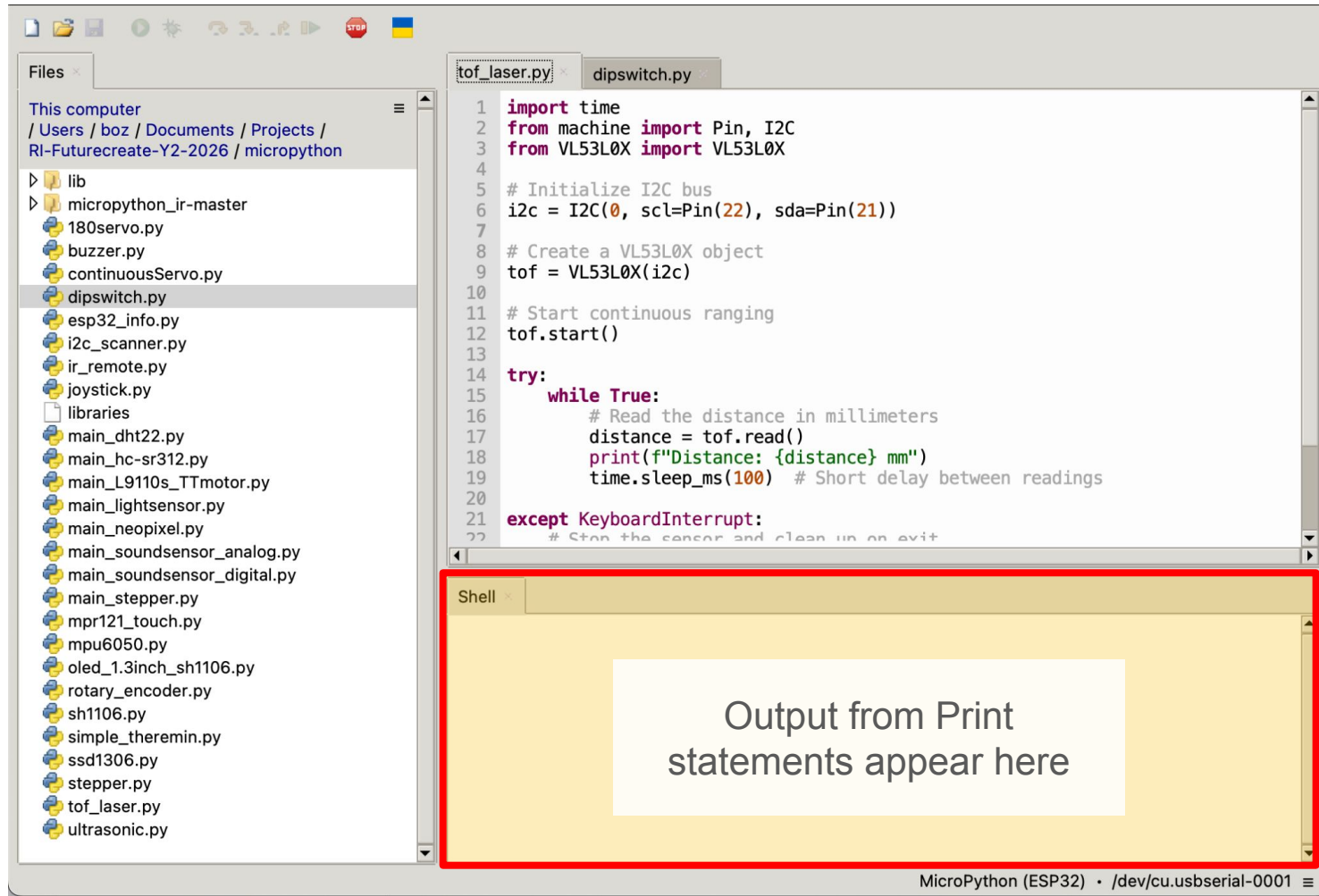
```
MPY: soft reboot
MicroPython v1.26.1 on 2025-09-11; Generic ESP32 module with ESP32
Type "help()" for more information.
>>>
```

A red box highlights the Shell window, and a white box within it contains the text 'When ESP32 is connected...'.

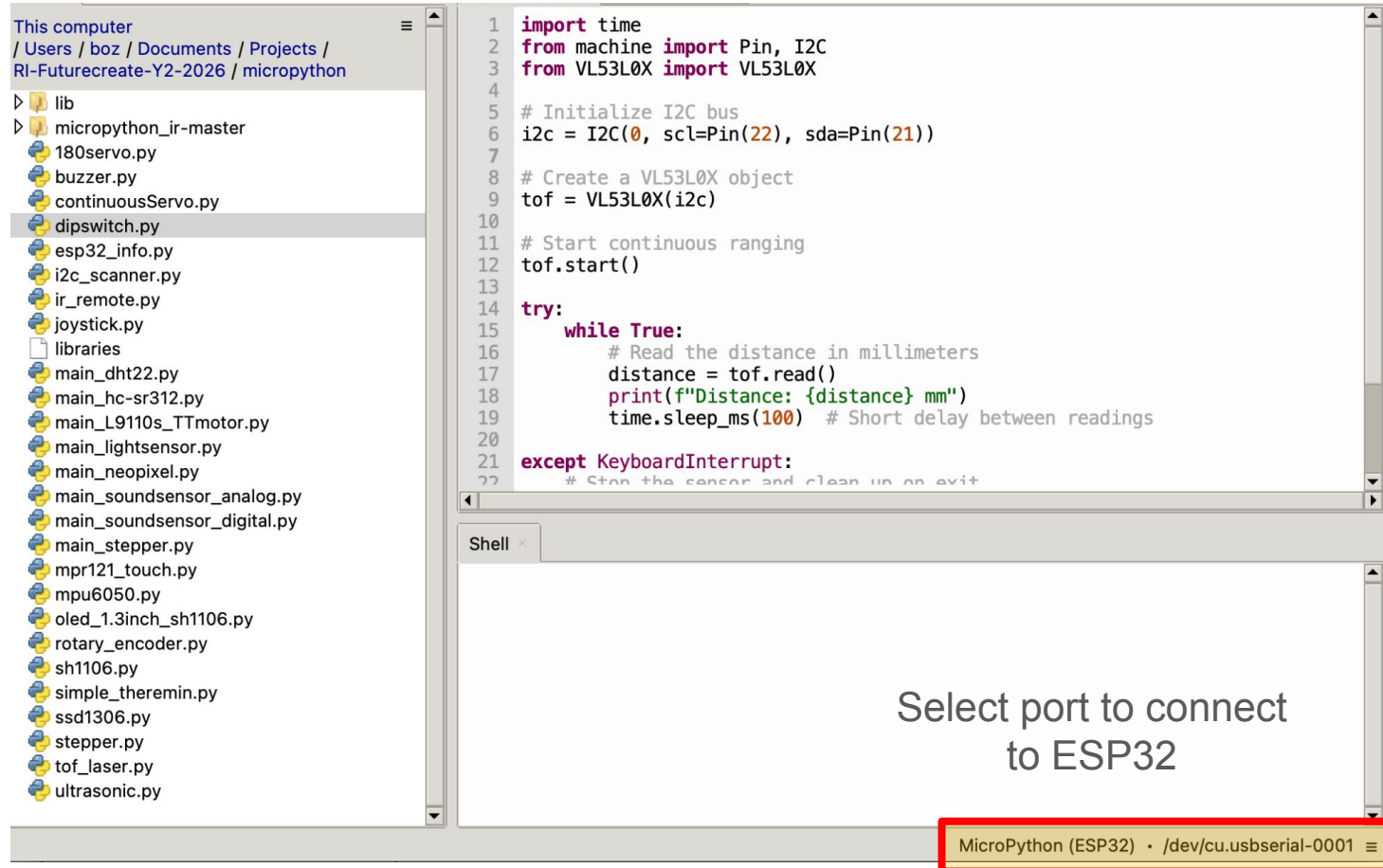
Intro to Thonny IDE



Intro to Thonny IDE



Intro to Thonny IDE



Intro to Thonny IDE

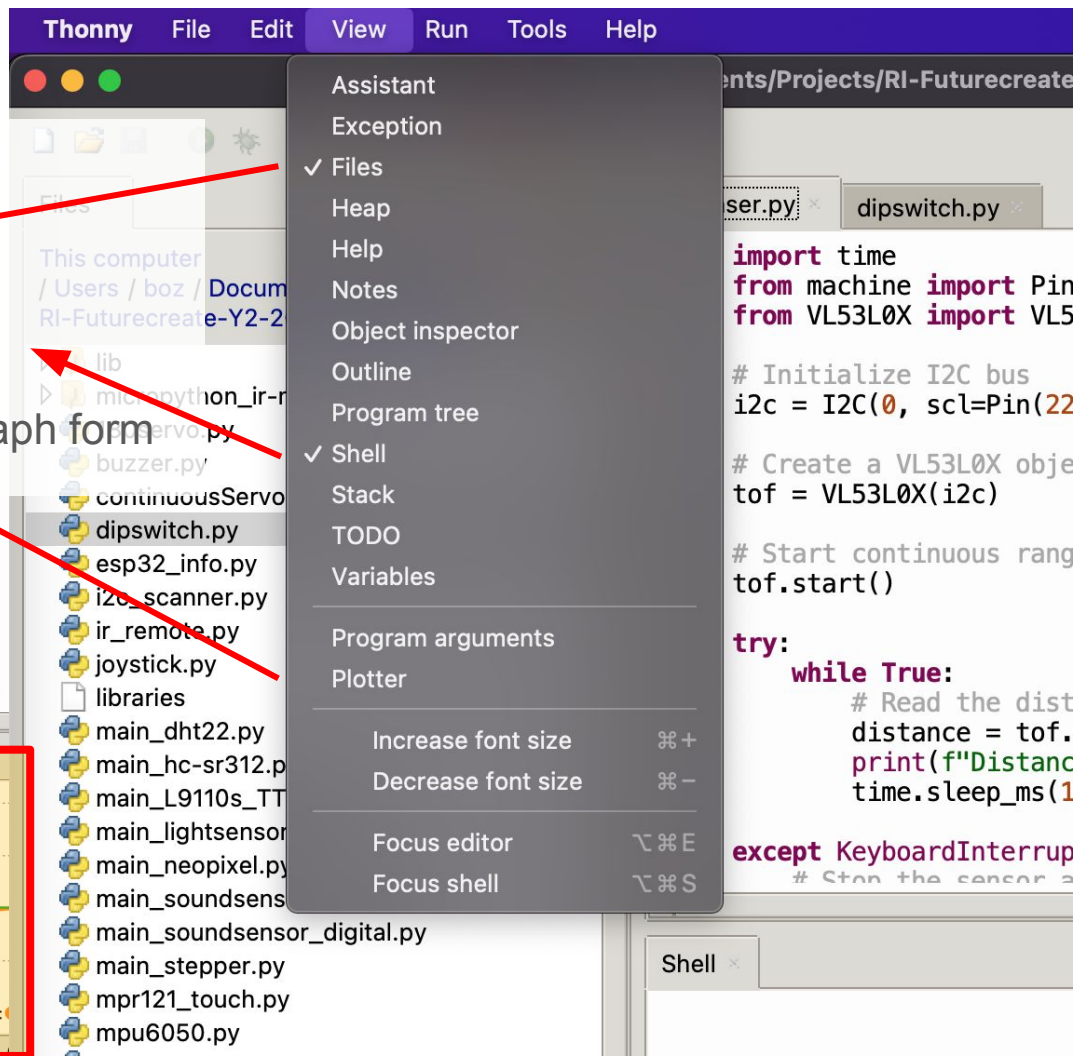
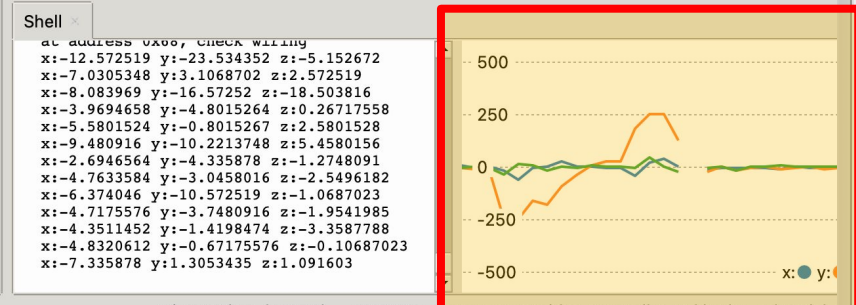
Other features:

Files - Local files

Shell - Output from print statements

Plotter - display printed values in graph form

```
16 az = accel["z"]
17 #print("x: " + str(ax) + " y: " + str(ay) + " z: " + str(az))
18
19 # Gyroscope Data
20 gyro = mpu.read_gyro_data() # read the gyro [deg/s]
21 gX = gyro["x"]
22 gY = gyro["y"]
23 gZ = gyro["z"]
24 print("x: " + str(gX) + " y:" + str(gY) + " z:" + str(gZ))
25
26 # Rough Temperature
27 temp = mpu.read_temperature() # read the device temperature [degC]
28 # print("Temperature: " + str(tempn) + "°C")
```

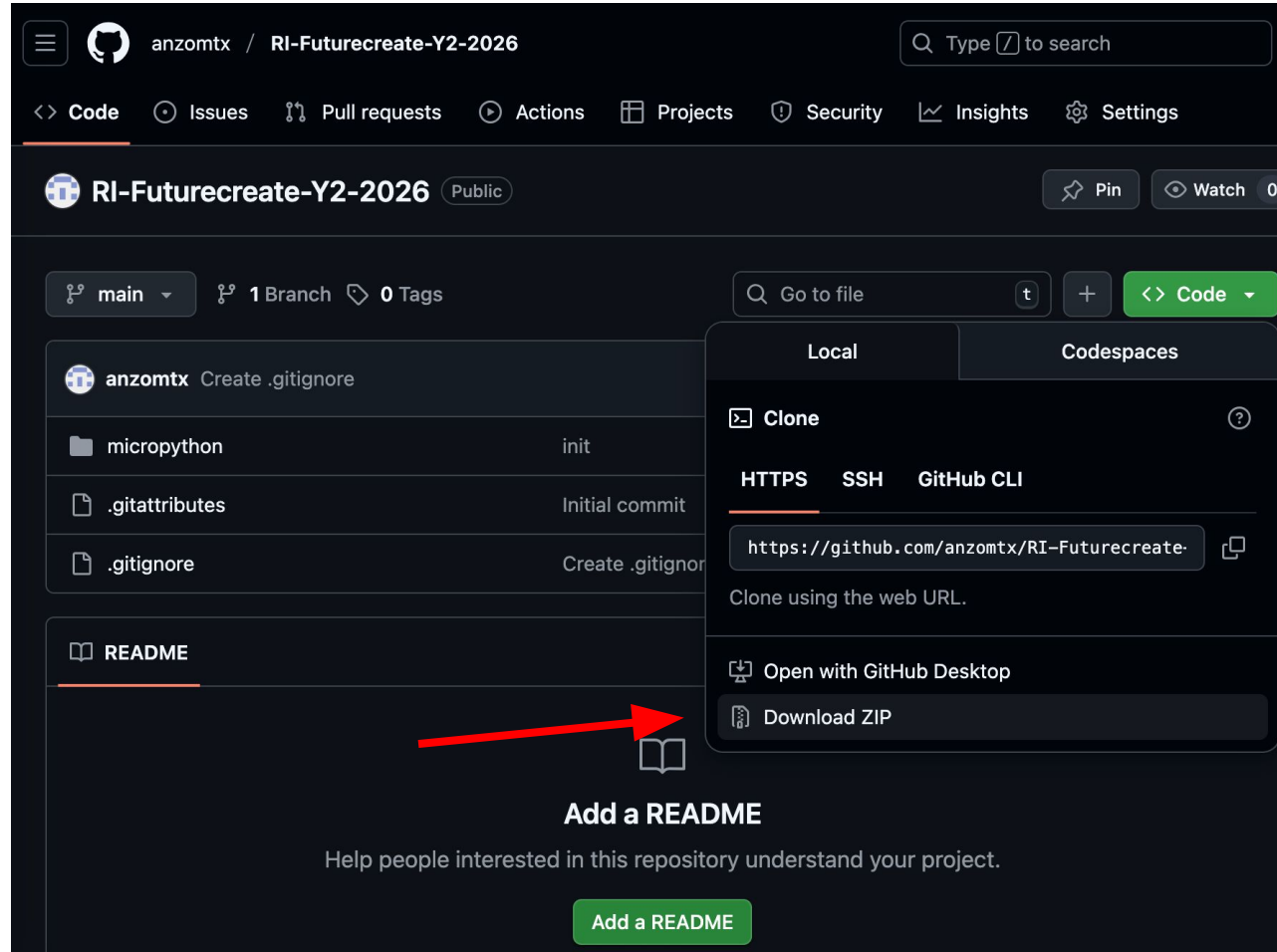


Setup files

Download code:

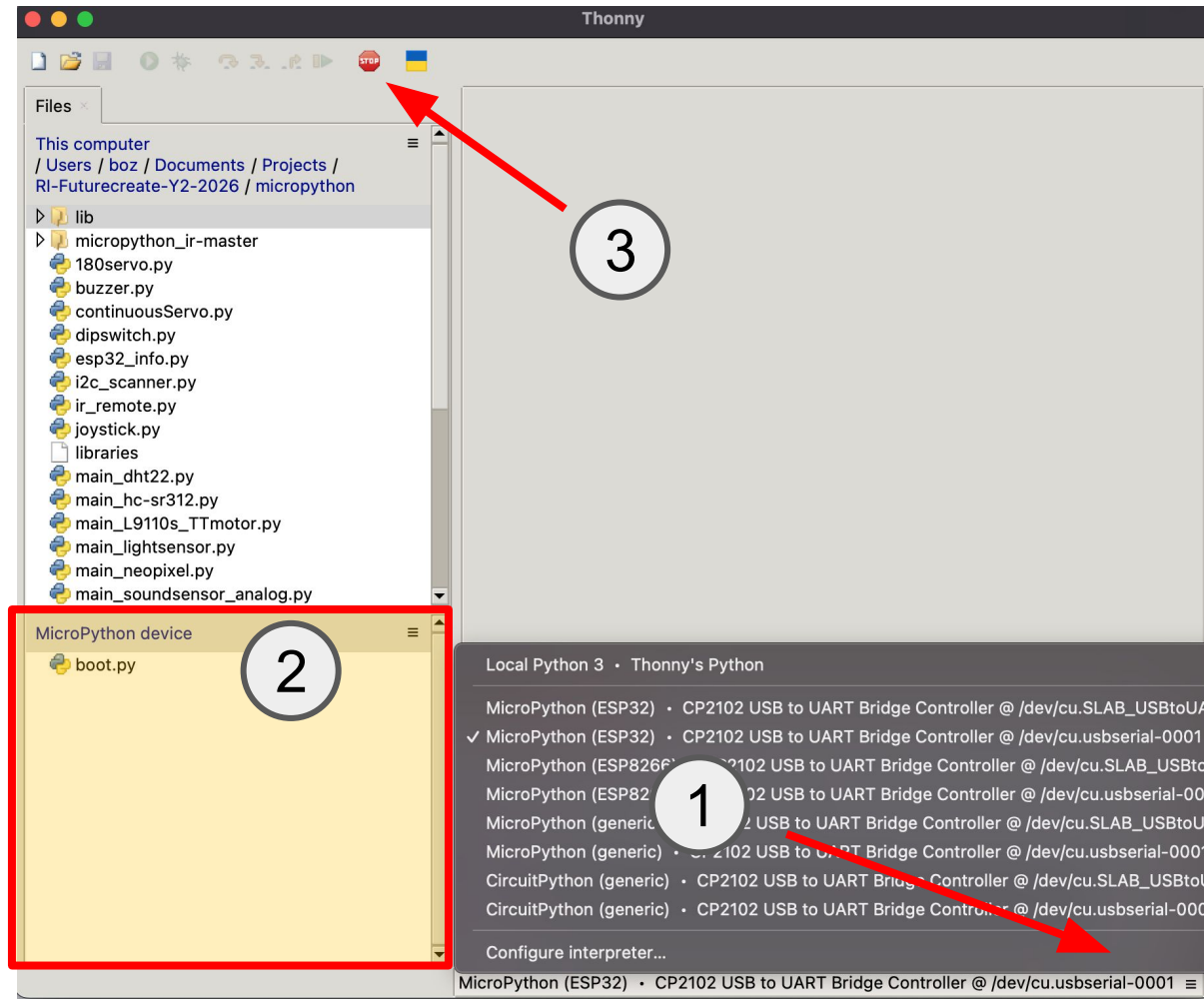
bit.ly/3mis4b

Unzip and move to
home folder



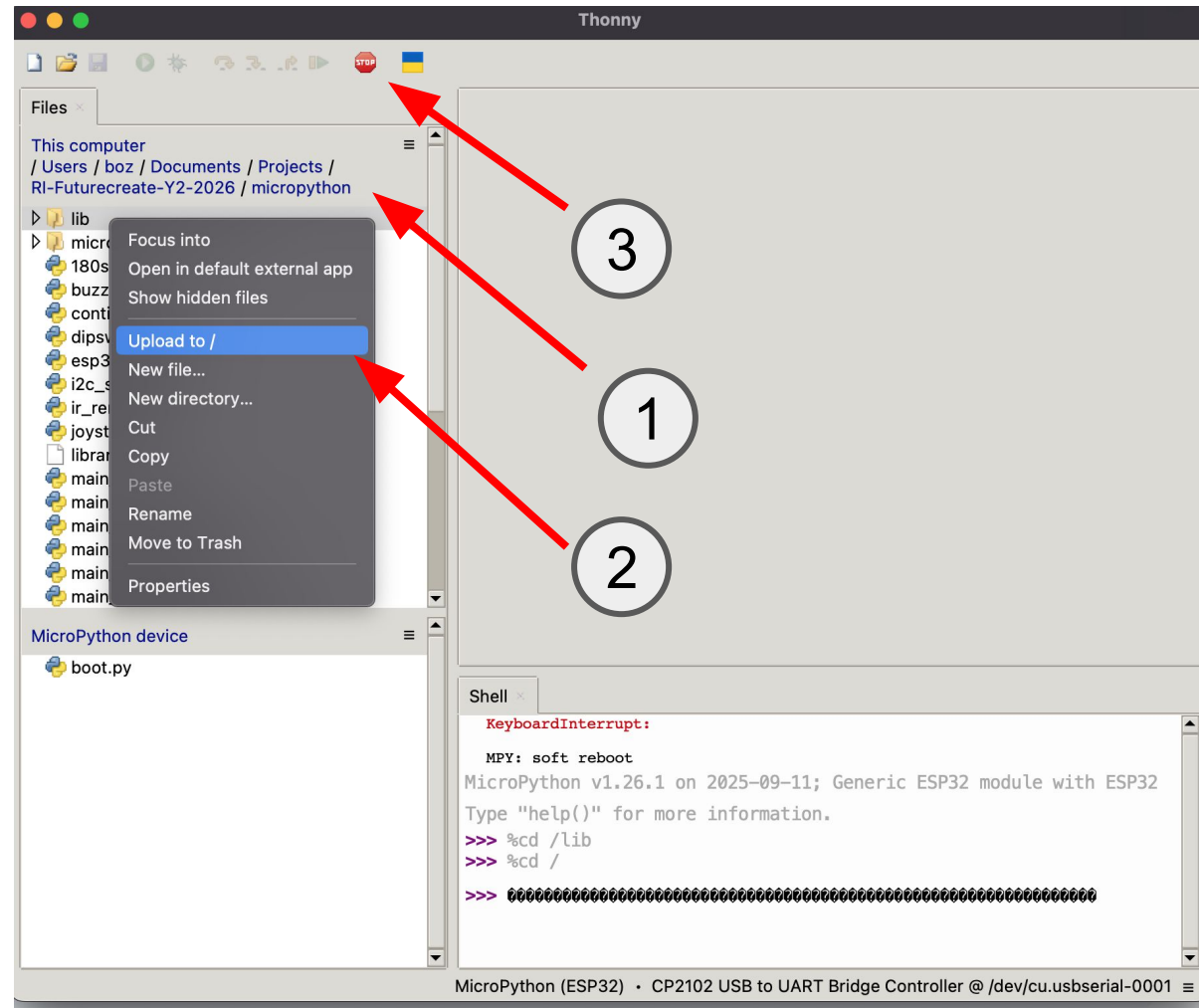
Setup files

1. Select correct port and connect to ESP32
2. If ESP32 is connected, Micropython device's filesystem will appear.
3. You might need to press restart button to reset ESP32



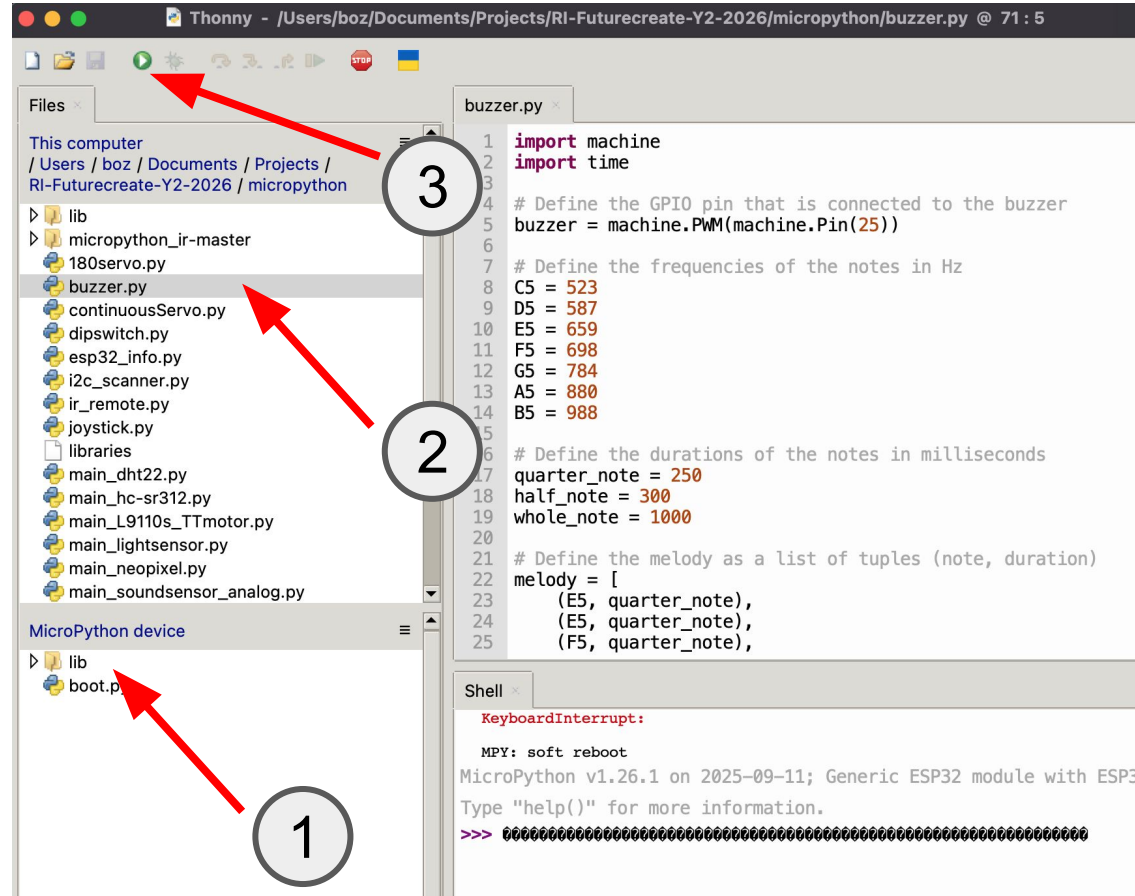
Setup files

1. Ensure local working directory is showing your downloaded code
2. Right-click on 'lib' folder and choose **Upload to /**
3. If you encounter errors, press restart button and try again.

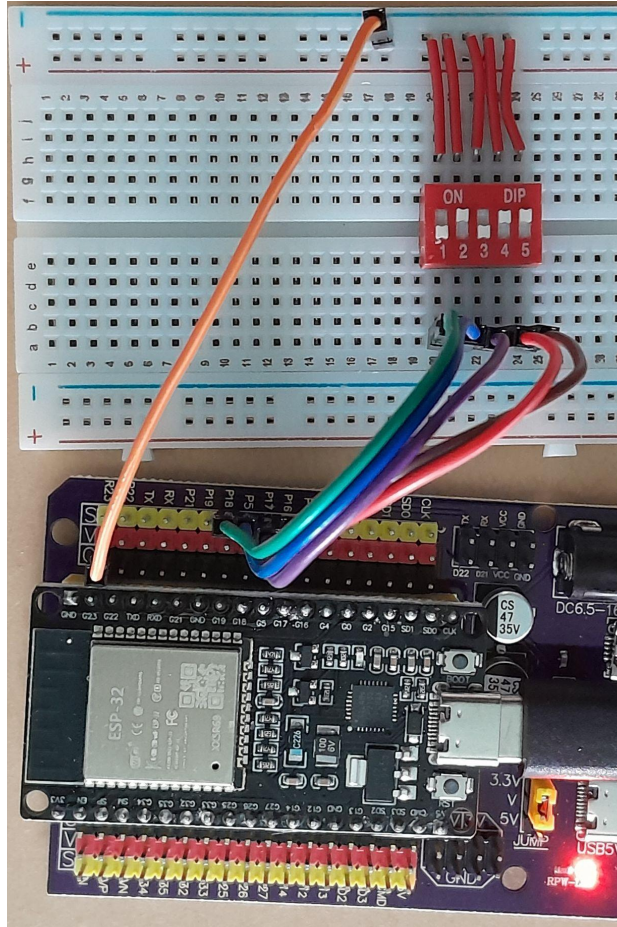
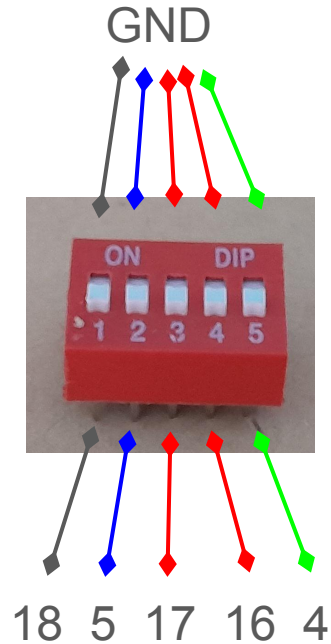


Setup files

1. You should see a 'lib' folder on your ESP32 device's filesystem
2. To open sample code, double click on a file on your local directory
3. Click on run button to execute the code on your ESP32.



Dip switch (switch.py)



switch.py

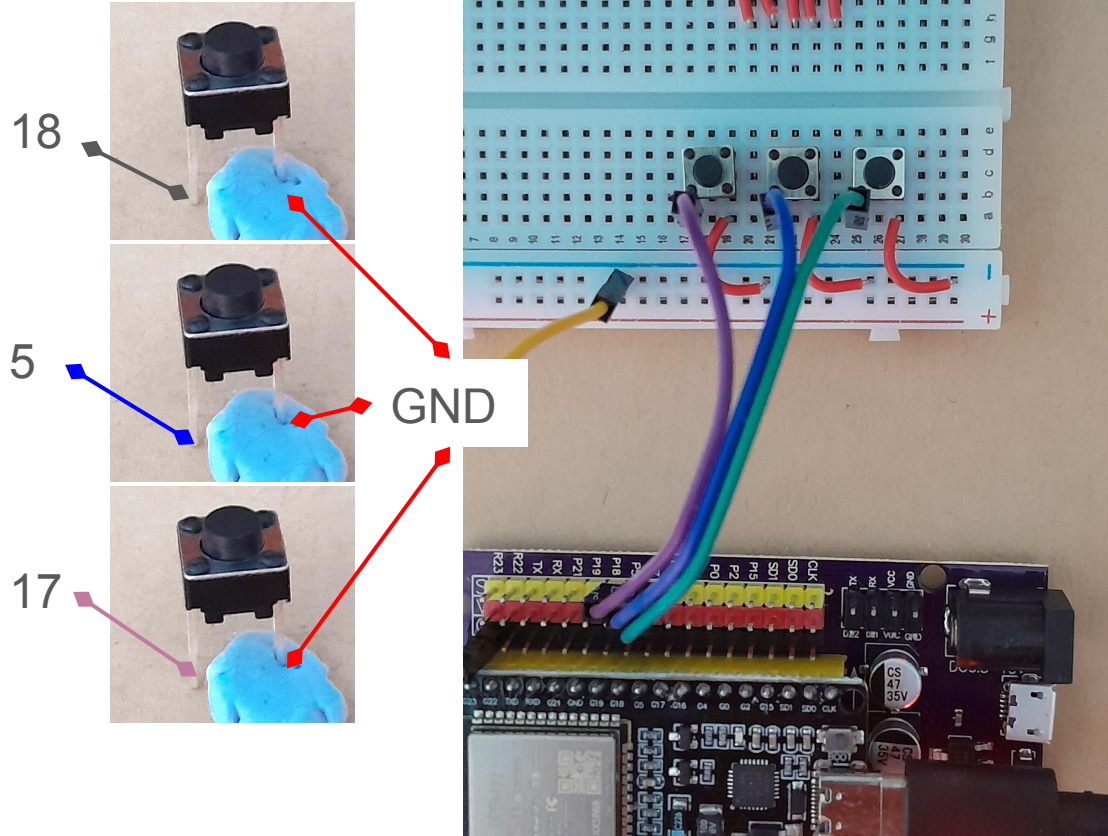
```
1 from machine import Pin
2 import time
3
4 # Define the number of positions and the GPIO pins for ea
5
6 SWITCH_PINS = [18, 5, 17, 16, 4] # Update these based on
7 POSITION_NUM = len(SWITCH_PINS)
8
9 ON = 0
10 OFF = 1
11
12 switches = []
13 for pin_num in SWITCH_PINS:
14     switches.append(Pin(pin_num, Pin.IN, Pin.PULL_UP))
15
16 print("DIP Switch Reader Started...")
17
18 try:
19     while True:
20         # Read and print the state of each switch position
21         for i in range(POSITION_NUM):
22             state = switches[i].value()
23             if state == ON :
```

Shell

```
Position 3: OFF
Position 4: ON
Position 5: ON
```

```
Position 1: OFF
Position 2: ON
Position 3: OFF
Position 4: ON
Position 5: ON
```

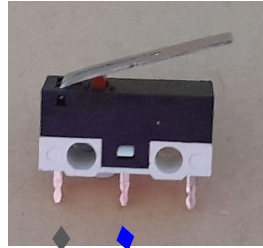
Push buttons (switch.py)



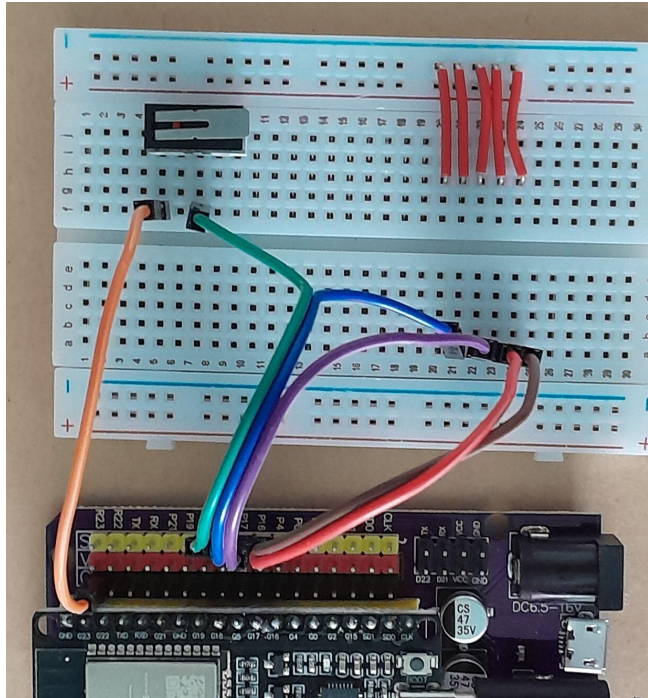
```
switch.py x
1 from machine import Pin
2 import time
3
4 # Define the number of positions and the GPIO pin
5
6 SWITCH_PINS = [18, 5, 17] # Update these based
7 POSITION_NUM = len(SWITCH_PINS)
8
9 ON = 0
10 OFF = 1
11
12 switches = []
13 for pin_num in SWITCH_PINS:
14     switches.append(Pin(pin_num, Pin.IN, Pin.PULL_UP))
15
16 print("DIP Switch Reader Started...")
17
18 try:
19     while True:
20         # Read and print the state of each switch
21         for i in range(POSITION_NUM):
22             state = switches[i].value()
```

```
Shell x
Position 3: OFF
Position 1: OFF
Position 2: OFF
Position 3: OFF
```

micro switch (switch.py)



18 GND



Note: the other wires on breadboard are not active as they are not connected to anything

switch.py ×

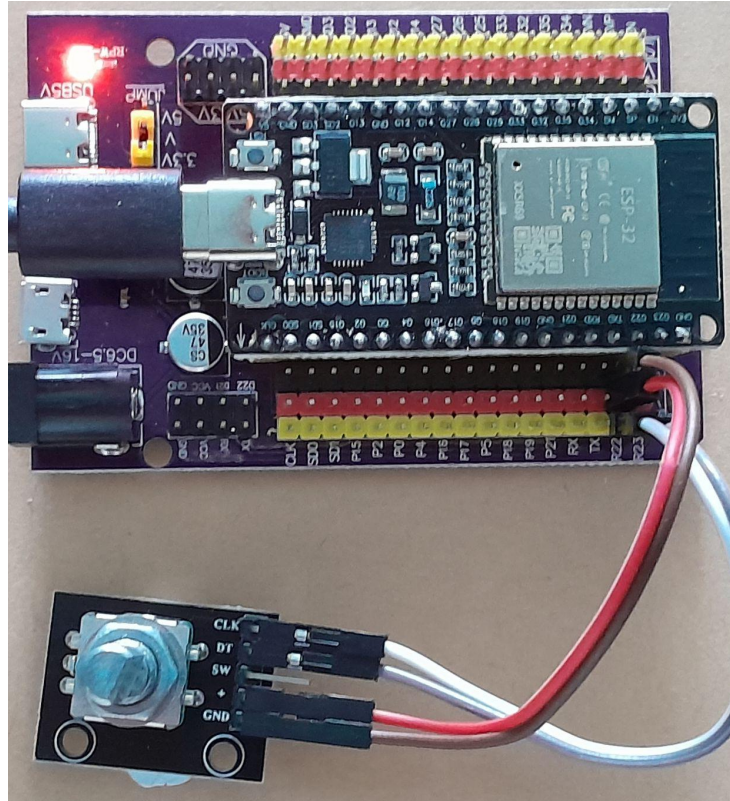
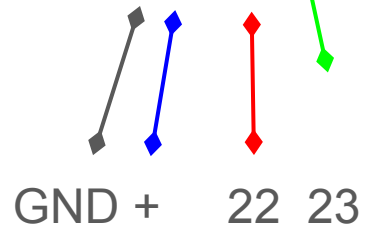
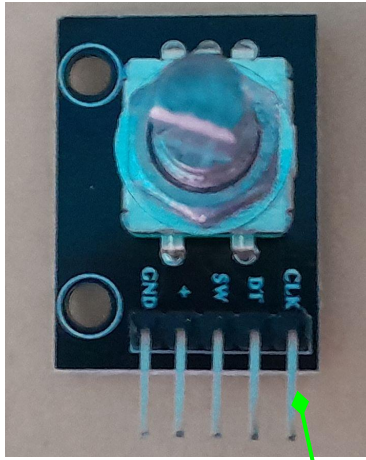
```
1 from machine import Pin
2 import time
3
4 # Define the number of positions and the GPIO pins for e
5
6 SWITCH_PINS = [18] # Update these based on your wiring
7 POSITION_NUM = len(SWITCH_PINS)
8
9 ON = 0
10 OFF = 1
11
12 switches = []
13 for pin_num in SWITCH_PINS:
14     switches.append(Pin(pin_num, Pin.IN, Pin.PULL_UP))
15
16 print("DIP Switch Reader Started...")
17
18 try:
19     while True:
20         # Read and print the state of each switch positi
21         for i in range(POSITION_NUM):
22             state = switches[i].value()
```

Shell ×

Position 1: ON

Position 1: ON

Rotary encoder (rotary_encoder.py)



```
rotary_encoder.py
1 from machine import Pin
2 from rotary_irq_esp import RotaryIRQ
3
4 # Initialize the rotary encoder with internal pull-up
5 rotary_encoder = RotaryIRQ(
6     pin_num_clk=22, # connect to CLK
7     pin_num_dt=23,  # connect to DT
8     pull_up=True    # Enable internal pull-up res
9 )
10
11 # Optional: set boundaries and wrapping
12 rotary_encoder.set(min_val=0, max_val=20, range_mod
13
14 last_value = rotary_encoder.value()
15
16 while True:
17     current_value = rotary_encoder.value()
18
19     if current_value != last_value:
20         print("Counter value:", current_value)
21
22         if current_value > last_value:
```

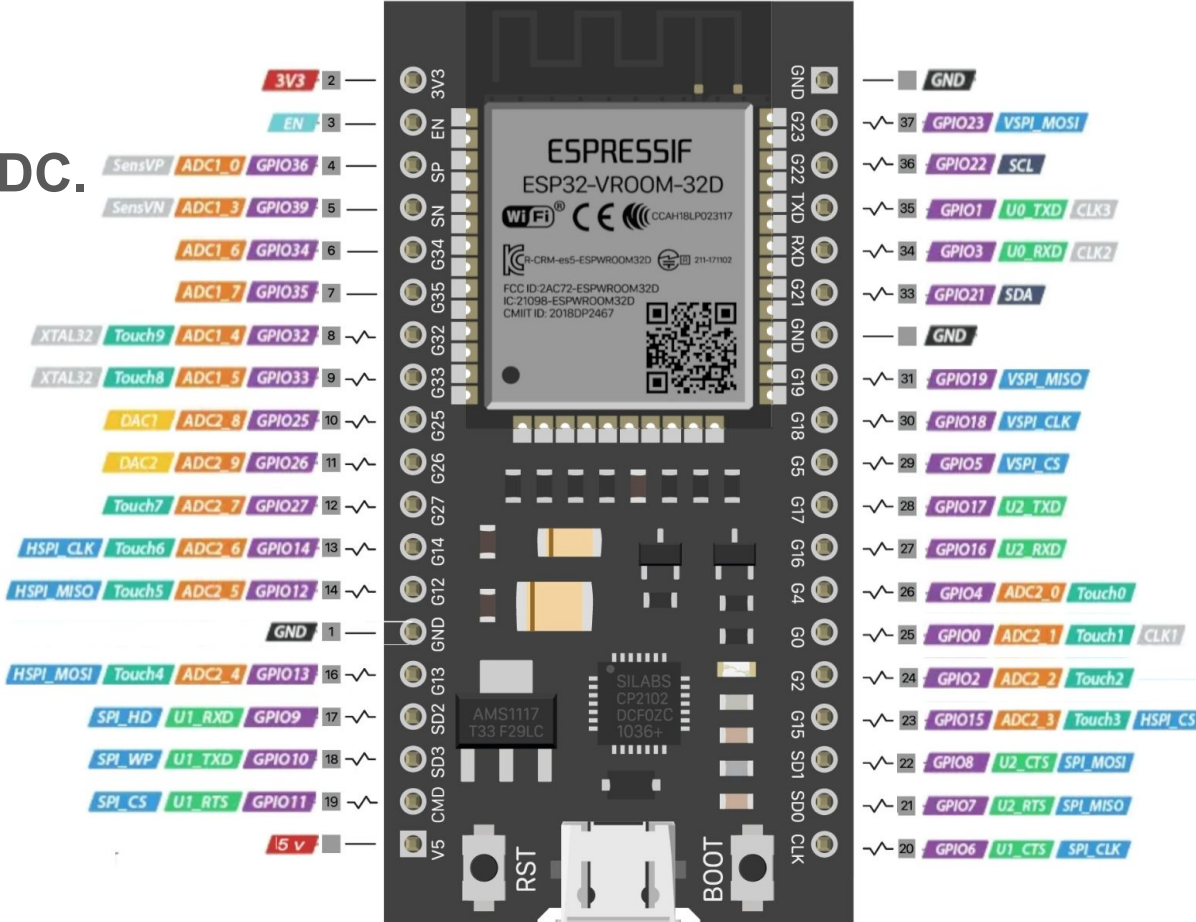
```
Shell
Counter value: 1
Direction: decreased
Counter value: 0
Direction: decreased
Counter value: 20
Direction: increased
Counter value: 19
Direction: decreased
Counter value: 18
Direction: decreased
```

Working with ESP32

Only select pins support ADC.

ADC pins:

- 2, 4
- 12, 13, 14
- 25, 26, 27
- 32, 33, 34, 35

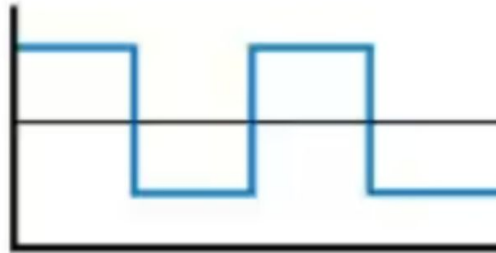


ADC (Analog to Digital) Pins

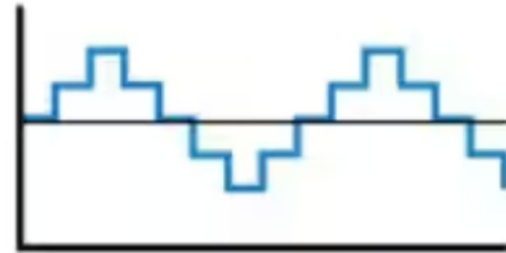
Digital world of computers only understand binary numbers (0s and 1s)

An ADC pin converts analog signals (like sound, light, temperature, or pressure) into digital signals

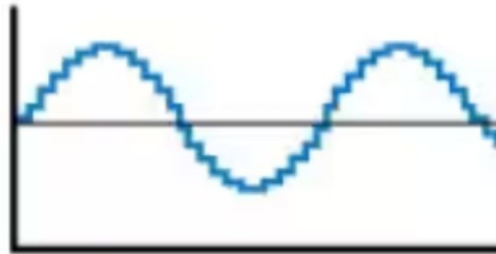
‘Bits’ represent resolution, more is better



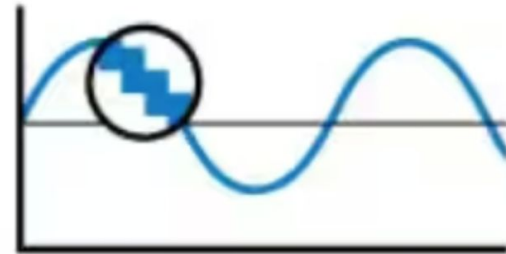
1-bit



2-bit



4-bit



16-bit

Joystick (joystick.py)

joystick.py

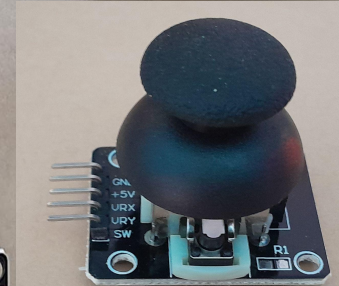
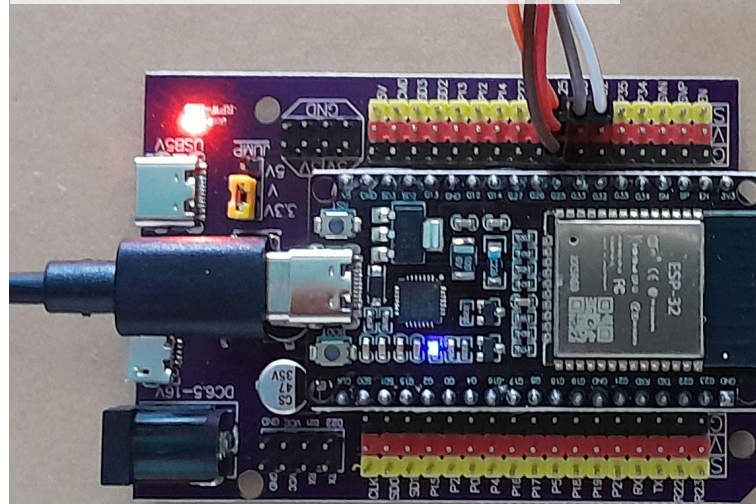
```
1 from DIYables_MicroPython_Joystick import Joystick
2 from machine import Pin, ADC
3 import time
4
5 VRX_PIN = 33 # connect to VRX pin of joystick
6 VRY_PIN = 32 # connect to VRY pin of joystick
7 SW_PIN = 25 # connect to SW pin of joystick
8
9 adc_vrx = ADC(Pin(VRX_PIN))
10 adc_vry = ADC(Pin(VRY_PIN))
11 # Set the ADC width (resolution) to 12 bits
12 adc_vrx.width(ADC.WIDTH_12BIT)
13 adc_vry.width(ADC.WIDTH_12BIT)
14 # Set the attenuation to 11 dB, allowing input range up
15 adc_vrx.atten(ADC.ATTN_11DB)
16 adc_vry.atten(ADC.ATTN_11DB)
17
18 joystick = Joystick(pin_x=VRX_PIN, pin_y=VRY_PIN, pin_sw=SW_PIN)
19
20 # Configure the debounce time if necessary (default is 100ms)
21 joystick.set_debounce_time(100) # debounce time set to 100ms
22
```

Shell

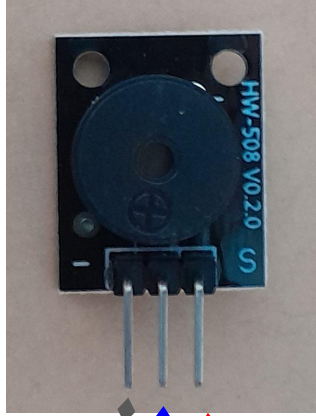
```
Joystick Position - X: 0, Y: 1921, pressed count: 4
Joystick Position - X: 0, Y: 1945, pressed count: 4
Joystick Position - X: 0, Y: 1930, pressed count: 4
Joystick Position - X: 0, Y: 1933, pressed count: 4
Joystick Position - X: 0, Y: 1931, pressed count: 4
Joystick Position - X: 0, Y: 1931, pressed count: 4
Joystick Position - X: 0, Y: 1931, pressed count: 4
Joystick Position - X: 0, Y: 1931, pressed count: 4
Joystick Position - X: 0, Y: 1930, pressed count: 4
Joystick Position - X: 0, Y: 1922, pressed count: 4
Joystick Position - X: 0, Y: 1945, pressed count: 4
```

Joystick - ESP32

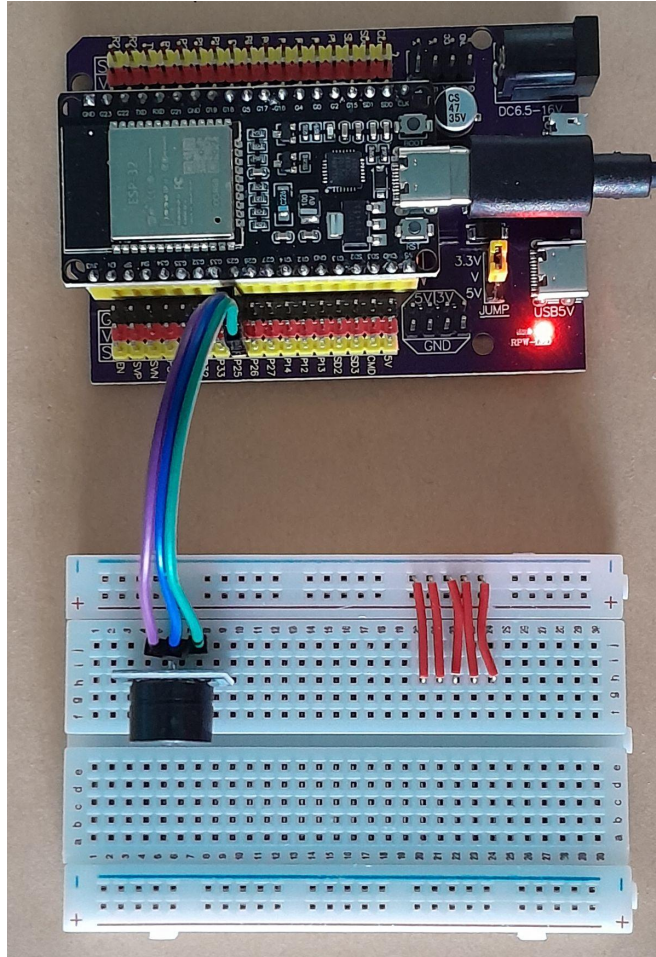
GND	GND (black)
+5V	VCC (red)
VRX	33
VRY	32
SW	25



Making simple sounds with the buzzer (buzzer.py)



GND + 25



buzzer.py ×

```
1 import machine
2 import time
3
4 # Define the GPIO pin that is connected to the buzzer
5 buzzer = machine.PWM(machine.Pin(25))
6
7 # Define the frequencies of the notes in Hz
8 C5 = 523
9 D5 = 587
10 E5 = 659
11 F5 = 698
12 G5 = 784
13 A5 = 880
14 B5 = 988
15
16 # Define the durations of the notes in milliseconds
17 quarter_note = 250
18 half_note = 300
19 whole_note = 1000
20
21 # Define the melody as a list of tuples (note, duration)
22 melody = []
```

Shell ×

```
>>> %Run -c $EDITOR_CONTENT
```


Session 1-3 prompt

Design something that turns everyday moments into a show.

If you are dry on ideas...

- A wacky music instrument control panel
- A control panel that works with human movement
- A control panel that looks or acts like a creature

